

VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF APPLIED SCIENCE



Machine Learning (CO3117)

Assignment

Machine Learning with Image Data

Food Ingredient Classification

Instructor(s): Dr.Trường Vĩnh Lân
Class: CC01



No.	Fullname	Student ID	Completion Rate	Tasks
1	Đinh Tiến (Leader)	2353174	100%	Feature Extraction, Conclusion, Future Work, Appendix & Source Code
2	Nguyễn Thành Tài	2353062	100%	Introduction, Dataset Description, Exploratory Data Analysis (EDA), Model Training
3	Nguyễn Minh Trực	2353264	100%	Data Preprocessing, Deep Learning Pipeline, Comparison: Traditional ML vs Deep Learning
4	Đinh An Huy	2352372	100%	Evaluation Metrics, Experimental Results, Advanced Analysis & Visualization, Discussion



TABLE OF CONTENTS

1	Introduction	3
2	Dataset Description	3
3	Exploratory Data Analysis (EDA)	4
3.1	Data Structure	4
3.2	Missing Values	5
3.3	Distribution Analysis	5
3.4	Class Imbalance	5
3.5	Image Analysis	6
4	Data Preprocessing	6
4.1	Label Cleaning	6
4.2	Duplicate Removal	7
4.3	Missing Data Handling	7
4.4	Class Filtering	7
4.5	Train/Validation/Test Split	8
4.6	Image Processing	8
4.7	Parallel Processing	8
5	Feature Extraction	9
5.1	CNN Features	9
5.1.1	ResNet50	9
5.1.2	EfficientNetB0	9
5.2	Transformer Features	9
5.2.1	Vision Transformer (ViT)	9
5.3	Output Files	10
6	Model Training	10
6.1	Models Used	10
6.2	Feature Scaling	11
6.3	Training Strategy	11
7	Evaluation Metrics	12



8	Experimental Results	13
8.1	Performance Table	13
8.2	Best Model	13
9	Advanced Analysis and Visualization	14
9.1	Feature Space Visualization	14
9.2	Confusion Matrix	15
9.3	Error Analysis	17
9.4	Model Comparison	18
10	Discussion	18
11	Deep Learning Pipeline	20
11.1	End-to-End Deep Learning Approach	20
11.2	Architecture Used	20
11.3	Training Process	21
11.4	Results	21
12	Comparison: Traditional ML vs Deep Learning	22
12.1	Accuracy Comparison	23
12.2	Speed and Complexity	23
12.3	Pros and Cons	24
13	Conclusion	25
14	Future Work	25

1 Introduction

Food ingredient classification from images is an important computer vision task with applications in dietary analysis, smart kitchen systems, and food management. Compared with general food recognition, ingredient classification is more difficult because many classes look visually similar and the number of samples per class is limited.

This project formulates the problem as a multi-class image classification task in which each image is assigned an ingredient label. The main focus is a traditional machine learning pipeline built on pretrained visual features, allowing the study to compare different feature extractors with lightweight classifiers.

The objectives are twofold: first, to build a complete workflow including dataset analysis, preprocessing, feature extraction, model training, and evaluation; second, to compare CNN-based and transformer-based feature representations. The dataset used is the **food-ingredients-dataset** from Hugging Face, which contains 6676 images and 316 ingredient classes.

Overall, this report evaluates whether pretrained features combined with classical classifiers such as Logistic Regression, Support Vector Machine, and Random Forest can provide strong performance, while also establishing a baseline for later comparison with the end-to-end deep learning pipeline.

2 Dataset Description

Source: This project uses the **Scuorese/food-ingredients-dataset** dataset available on Hugging Face. The dataset is designed for food ingredient recognition and provides image samples together with hierarchical textual labels. Because it is distributed through the Hugging Face ecosystem, it can be loaded directly into Python workflows and converted conveniently into tabular form for analysis and preprocessing.

Structure: The dataset contains four columns:

- **image:** the raw food ingredient image;
- **category:** the top-level food group label;
- **subcategory:** the intermediate label within each category;
- **ingredient:** the fine-grained ingredient label used as the main prediction target.

This hierarchical structure is useful because it supports analysis at different semantic levels. In particular, the `ingredient` column provides the most detailed label and is therefore the most challenging target for classification.

Dataset Size: The raw dataset contains 6676 samples and 4 columns. Each row represents one image-label pair, where the image is stored together with its category, subcategory, and ingredient annotations. This sample size is sufficient for experimentation, but it is still relatively limited for a fine-grained classification task with hundreds of target classes.

Number of Classes: The class distribution can be summarized at three levels:

- 12 unique `category` classes;
- 28 unique `subcategory` classes;
- 316 unique `ingredient` classes.

The large number of ingredient classes compared with the total number of images shows that this is a high-granularity classification problem. On average, each ingredient class has only a limited number of samples, which increases the difficulty of both training and generalization.

Implications for the Task: The dataset is well suited for evaluating pretrained visual representations because the fine-grained labels require the model to capture subtle differences between visually similar ingredients. However, the combination of many classes and relatively few samples per class also creates challenges such as class imbalance, limited intra-class diversity, and a higher risk of overfitting. These issues motivate the exploratory data analysis and preprocessing steps presented in the following sections.

3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis was conducted to understand the dataset structure, assess data quality, examine class distributions, and inspect image properties before preprocessing and model development. This stage is important because it reveals potential issues such as imbalance, missing values, and inconsistent image sizes that may affect downstream training performance.

3.1 Data Structure

The raw dataset contains 6676 rows and 4 columns: `category`, `subcategory`, `ingredient`, and `image`. The first three columns are string-based labels, while the `image` column stores image objects. This confirms that the dataset is organized as a labeled image collection with hierarchical semantic annotations.

At the class level, the dataset contains 12 categories, 28 subcategories, and 316 ingredient classes. This shows that the task becomes increasingly fine-grained from category to ingredient level, with ingredient classification being the most difficult stage because it contains the largest number of classes.

3.2 Missing Values

The missing-value analysis showed that all four columns are complete. The counts of missing values for `category`, `subcategory`, `ingredient`, and `image` are all zero, corresponding to 0.0% missing data in every column.

This result indicates that the raw dataset is structurally complete and does not require imputation. Nevertheless, checking missing values remains necessary because missing labels or missing image content would make supervised training unreliable. Since no missing entries were found, the dataset could proceed directly to duplicate checking and label cleaning.

3.3 Distribution Analysis

Distribution analysis was performed at three label levels. First, the category distribution provides a broad overview of how the dataset is divided across top-level food groups. Second, the top 15 most frequent subcategories were visualized to identify which mid-level classes dominate the dataset. Third, the top 20 most frequent ingredients were examined to better understand the concentration of samples at the target-label level.

These plots show that the dataset is not uniformly distributed. Some categories and ingredient classes appear much more frequently than others, which is common in real-world food datasets. This uneven distribution suggests that a naive classifier could become biased toward frequent classes if class imbalance is not considered during preprocessing and evaluation.

3.4 Class Imbalance

Class imbalance becomes more pronounced at the ingredient level, where 316 classes share a total of only 6676 images. A histogram of class frequencies and a boxplot of ingredient sample counts reveal that many ingredients have relatively few images, while only a smaller subset of ingredients has high representation.

This imbalance has important implications for model training. Minority classes are more difficult to learn reliably, and severe imbalance can reduce macro-level performance even when overall accuracy seems acceptable. This observation justifies the later preprocessing step that

removes classes with fewer than 10 samples in order to create a more stable training set.

3.5 Image Analysis

Visual inspection of randomly sampled images confirms that the dataset contains substantial variation in background, lighting, composition, scale, and viewpoint. While this diversity is useful for realism, it also increases the learning difficulty because visually similar ingredients may appear under very different imaging conditions.

The image dimension analysis shows that width and height are highly variable. The average width is approximately 1151 pixels and the average height is approximately 1089 pixels, while the smallest images are around 173×170 and the largest images reach up to 6720×5286 . The median size is close to 1000×1000 , but the wide spread confirms that the dataset does not have a uniform resolution.

These findings justify resizing all images to 224×224 during preprocessing. A fixed resolution is necessary to ensure compatibility with pretrained CNN and ViT backbones, simplify batch processing, and maintain consistency across the entire feature extraction and training pipeline.

4 Data Preprocessing

This section describes the preprocessing pipeline applied to the dataset before model training. The main goals are to standardize class names, remove noisy or duplicated samples, handle incomplete records, and prepare the images in a format suitable for reliable training and evaluation.

4.1 Label Cleaning

Raw class labels often contain inconsistent capitalization and separator styles, which can cause identical classes to be treated as different categories. To ensure consistency, all labels were first converted to lowercase. In addition, spaces and slashes were replaced with underscore-style separators so that each class name could be used safely in file names and directory paths.

For example, labels such as “Red Apple”, “red/apple”, and “RED APPLE” were normalized into a single standardized form. This step reduces ambiguity and guarantees that the mapping from labels to folders remains consistent throughout the preprocessing and training stages.

4.2 Duplicate Removal

Duplicate images can bias training, inflate evaluation scores, and reduce the effective diversity of the dataset. To address this issue, a hash-based image comparison method was used. For each image, a hash value was computed and compared against previously processed samples. Images with identical hash values were treated as duplicates and removed from the dataset.

The total number of removed duplicates was recorded during preprocessing. In this project, 856 duplicate samples were removed. Reporting this quantity is important because it gives a direct indication of dataset redundancy and helps document how much cleaning was required before model development.

4.3 Missing Data Handling

Rows with missing labels or missing image files were dropped from the dataset. Samples without labels cannot be used in supervised learning, while records with unavailable or corrupted image paths prevent proper feature extraction and model input generation.

The missing-data statistics showed that the columns `extttcategory`, `extttsubcategory`, `exttingredient`, and `extttimage` each contained 0 missing values. Therefore, the dataset remained complete at this stage, with 5820 samples after handling missing values.

This decision is justified because keeping incomplete samples would introduce noise and additional uncertainty into the training process. Removing such rows ensures that every remaining sample contains both a valid image and a valid target label, which is essential for reproducible experiments.

4.4 Class Filtering

After cleaning, classes with fewer than 10 samples were removed. Very small classes do not provide enough variation for the model to learn stable patterns and may also cause unreliable train/validation/test splits. After this minority-class filtering step, the dataset contained 5811 samples.

This filtering step helps avoid training issues such as severe class imbalance, unstable metrics, and poor generalization for underrepresented categories. By retaining only classes with at least 10 samples, the dataset becomes more suitable for stratified splitting and downstream learning.

4.5 Train/Validation/Test Split

The cleaned dataset was divided into training, validation, and test subsets using a ratio of 70%/15%/15%. The training set is used for parameter learning, the validation set is used for model selection and hyperparameter tuning, and the test set is reserved for the final performance evaluation. In the final split, the training set contained 4067 samples, the validation set contained 872 samples, and the test set contained 872 samples.

To preserve the class distribution across all subsets, stratified sampling was applied during the split. This is especially important for multi-class image datasets because it prevents rare but valid classes from being concentrated in only one subset and ensures that evaluation results remain representative of the overall data distribution.

4.6 Image Processing

All images were converted to RGB format in order to standardize the number of channels across the dataset. This avoids incompatibilities caused by grayscale, RGBA, or other image modes that may appear in raw data collections.

Next, each image was resized to 224×224 pixels, which is a common input resolution for convolutional neural networks. After preprocessing, the images were saved into structured folders organized by split and class label. This directory structure simplifies dataset loading and makes the training pipeline easier to reproduce and maintain.

4.7 Parallel Processing

Because preprocessing may involve thousands of images, the pipeline was accelerated using multi-threading with `ThreadPoolExecutor`. Independent operations such as file loading, RGB conversion, resizing, hash computation, and file saving can be processed concurrently for multiple images.

This parallel design significantly improves preprocessing speed compared with a purely sequential pipeline, especially when disk I/O is a major bottleneck. As a result, the total preparation time is reduced, making the workflow more practical for large-scale experiments and repeated dataset updates.

5 Feature Extraction

Deep feature extraction is a critical stage in our pipeline, where high-dimensional image data is converted into compact numerical vectors. These vectors capture the essential visual characteristics of food ingredients, serving as the primary input for the downstream classical machine learning classifiers.

5.1 CNN Features

We utilized two popular Convolutional Neural Network (CNN) architectures to extract spatial features: ResNet50 and EfficientNetB0. Both models were loaded with weights pre-trained on the ImageNet dataset and used as frozen feature extractors.

5.1.1 ResNet50

The ResNet50 model is known for its residual learning framework, which allows the training of very deep networks. By discarding the classification head and appending a `GlobalAveragePooling2D` layer, we extracted 2048-dimensional feature vectors per image.

5.1.2 EfficientNetB0

EfficientNetB0 offers a more lightweight and efficient architecture through compound scaling of depth, width, and resolution. This backbone produced 1280-dimensional feature vectors, providing a more compact representation compared to ResNet50.

5.2 Transformer Features

In addition to CNNs, we implemented a Transformer-based approach using the Vision Transformer (ViT) to capture long-range spatial dependencies.

5.2.1 Vision Transformer (ViT)

We employed the `google/vit-base-patch16-224-in21k` checkpoint, which processes images as sequences of 16×16 patches. The representation of the entire image is captured through the special [CLS] token at position 0 of the last hidden state.

- **Implementation:** The pipeline was built using PyTorch and the `transformers` library.

- **Processing:** Images were scaled to a $[-1, 1]$ range and processed under `torch.no_grad()` to ensure efficiency.
- **Output:** The resulting feature vectors are 768-dimensional.

5.3 Output Files

All extracted features and their corresponding labels were saved in the `.npy` binary format to facilitate fast loading and consistent data transfer between group members.

Bảng 1: Summary of Extracted Feature Files

File Name	Content Description	Array Shape
<code>resnet_X_train.npy</code>	ResNet50 features (Train)	(4067, 2048)
<code>effnet_X_train.npy</code>	EfficientNetB0 features (Train)	(4067, 1280)
<code>vit_X_train.npy</code>	ViT [CLS] features (Train)	(4067, 768)
<code>vit_X_val.npy</code>	ViT [CLS] features (Validation)	(872, 768)
<code>vit_X_test.npy</code>	ViT [CLS] features (Test)	(872, 768)
<code>y_train.npy</code>	Integer-encoded labels (Train)	(4067,)
<code>y_test.npy</code>	Integer-encoded labels (Test)	(872,)

The `label_map.json` file was also generated to provide a consistent mapping between the integer indices and the actual ingredient names, ensuring accurate interpretation during the evaluation and visualization phases.

6 Model Training

This section follows the traditional machine learning stage of the pipeline, where pretrained deep features are used as the input for classical classifiers. Instead of training directly on raw images, each sample is first converted into a fixed-length feature vector by ResNet50, EfficientNetB0, or ViT. These representations are then used to train and compare several supervised learning models under the same train/validation/test setting.

6.1 Models Used

Three classifiers were used in this study: Logistic Regression, Support Vector Machine (SVM), and Random Forest.

Logistic Regression was selected as a strong linear baseline for multi-class classification. It is computationally efficient, easy to optimize, and often performs well when the input features are already informative, as is the case with pretrained deep embeddings.

Support Vector Machine was included because it is effective in high-dimensional feature spaces and can learn robust decision boundaries. In this project, the SVM uses an RBF kernel so that it can model non-linear relationships between extracted features and ingredient labels.

Random Forest was used as an ensemble-based non-linear classifier. By combining multiple decision trees, it can capture complex patterns without requiring gradient-based optimization. It also provides a useful contrast to the linear and margin-based models.

Together, these three models provide a balanced comparison across different learning strategies: linear classification, kernel-based classification, and tree-based ensemble learning.

6.2 Feature Scaling

Feature scaling was applied selectively depending on the classifier. For Logistic Regression and SVM, `StandardScaler` was used to standardize the extracted features before training. This step is important because both models are sensitive to differences in feature magnitude, and unscaled input can negatively affect optimization and decision boundaries.

In contrast, Random Forest was trained without feature scaling. Tree-based models split features based on threshold comparisons and therefore do not require standardized input in the same way as linear or kernel-based models. Avoiding unnecessary scaling for Random Forest keeps the pipeline simpler while preserving appropriate model behavior.

This design ensures that each classifier is trained under conditions that are consistent with standard machine learning practice.

6.3 Training Strategy

The training process was performed on the extracted feature vectors rather than on raw images. For each backbone representation, the training features were paired with the encoded labels and used to fit the three classifiers independently. The same experimental procedure was repeated for ResNet50 features, EfficientNetB0 features, and ViT features so that the influence of the feature extractor could be evaluated fairly.

The dataset split generated during preprocessing was kept unchanged throughout this stage. The training set contained 4067 samples, while the validation and test sets each contained 872 samples. The validation set was used for model selection and hyperparameter adjustment, while the test set was reserved for final evaluation.

This strategy creates a clear comparison framework across feature extractors:

- ResNet50 + classical classifiers;

- EfficientNetB0 + classical classifiers;
- ViT + classical classifiers.

By organizing the experiments in this way, the project can identify whether improvements come mainly from the feature representation or from the classifier choice. This also provides a strong baseline for the later evaluation and for comparison with the end-to-end deep learning pipeline.

7 Evaluation Metrics

The trained classifiers were evaluated using four standard multi-class classification metrics: accuracy, macro-averaged precision, macro-averaged recall, and macro-averaged F1-score. All metrics were computed on the held-out test set of 872 samples after fitting each model on the training features.

Accuracy is the overall fraction of correctly classified samples. It is the most intuitive of the four metrics, but it can be misleading when the dataset is imbalanced because it is dominated by the larger classes. For this reason, accuracy was complemented with three macro-averaged metrics that treat every class with equal weight.

Macro precision and macro recall are computed for each class and then averaged across all 315 ingredient classes. Per-class precision is the fraction of predictions for a given class that are correct, while per-class recall is the fraction of true-class samples that the model recovers. Because every class contributes equally to the average, these metrics are sensitive to performance on rare ingredients rather than only on common ones.

Macro F1-score is the harmonic mean of precision and recall, computed for each class and then averaged. It penalizes both false positives and false negatives, which is appropriate when neither error type can be ignored.

Macro averaging was chosen as the primary comparison criterion because the dataset is fine-grained and approximately balanced at the ingredient level, with around 19 to 21 samples per class. A weighted average would be dominated by the largest classes and would not reveal whether a classifier handles low-support categories well, while the macro variant directly captures this behavior. The same metric set was applied uniformly to all nine model–feature combinations to ensure that the results in Section 8 are directly comparable.

8 Experimental Results

This section reports the quantitative performance of all nine model–feature combinations on the test set. Each result was generated by training a classifier on the corresponding training feature matrix, applying feature scaling where required as described in Section 6.2, and evaluating the predictions on the test split using the metrics introduced in Section 7.

8.1 Performance Table

Table 2 summarizes the test-set performance of every combination. The full numerical results are also stored in `ml_results_all_features.csv` for reproducibility. Each row corresponds to a single classifier trained on a specific feature representation, and each column reports one of the four evaluation metrics.

Bảng 2: Test-set performance of all model–feature combinations.

Model	Feature	Accuracy	Precision	Recall	F1
Logistic Regression	ResNet50	0.439	0.448	0.434	0.418
SVM (RBF)	ResNet50	0.384	0.393	0.375	0.352
Random Forest	ResNet50	0.317	0.296	0.311	0.279
Logistic Regression	EfficientNetB0	0.476	0.484	0.470	0.456
SVM (RBF)	EfficientNetB0	0.415	0.425	0.407	0.387
Random Forest	EfficientNetB0	0.298	0.278	0.290	0.262
Logistic Regression	ViT	0.609	0.619	0.604	0.593
SVM (RBF)	ViT	0.558	0.575	0.549	0.537
Random Forest	ViT	0.486	0.485	0.479	0.458

The table reveals a clear three-way ranking. ViT-based features dominate every metric, EfficientNetB0 features come second on linear and kernel models, and ResNet50 features fall slightly behind EfficientNet under the same classifiers. Random Forest is the weakest classifier in every feature setting, which suggests that tree-based splits do not exploit the joint structure of the dense embedding vectors as effectively as linear or kernel methods.

8.2 Best Model

The best combination among all nine settings is Logistic Regression trained on ViT features, which achieved an accuracy of 0.6089, a precision of 0.6192, a recall of 0.6037, and an F1-score of 0.5929. This combination outperforms every other configuration on every metric and is

therefore selected as the reference model for the confusion matrix and error analysis in Section 10.

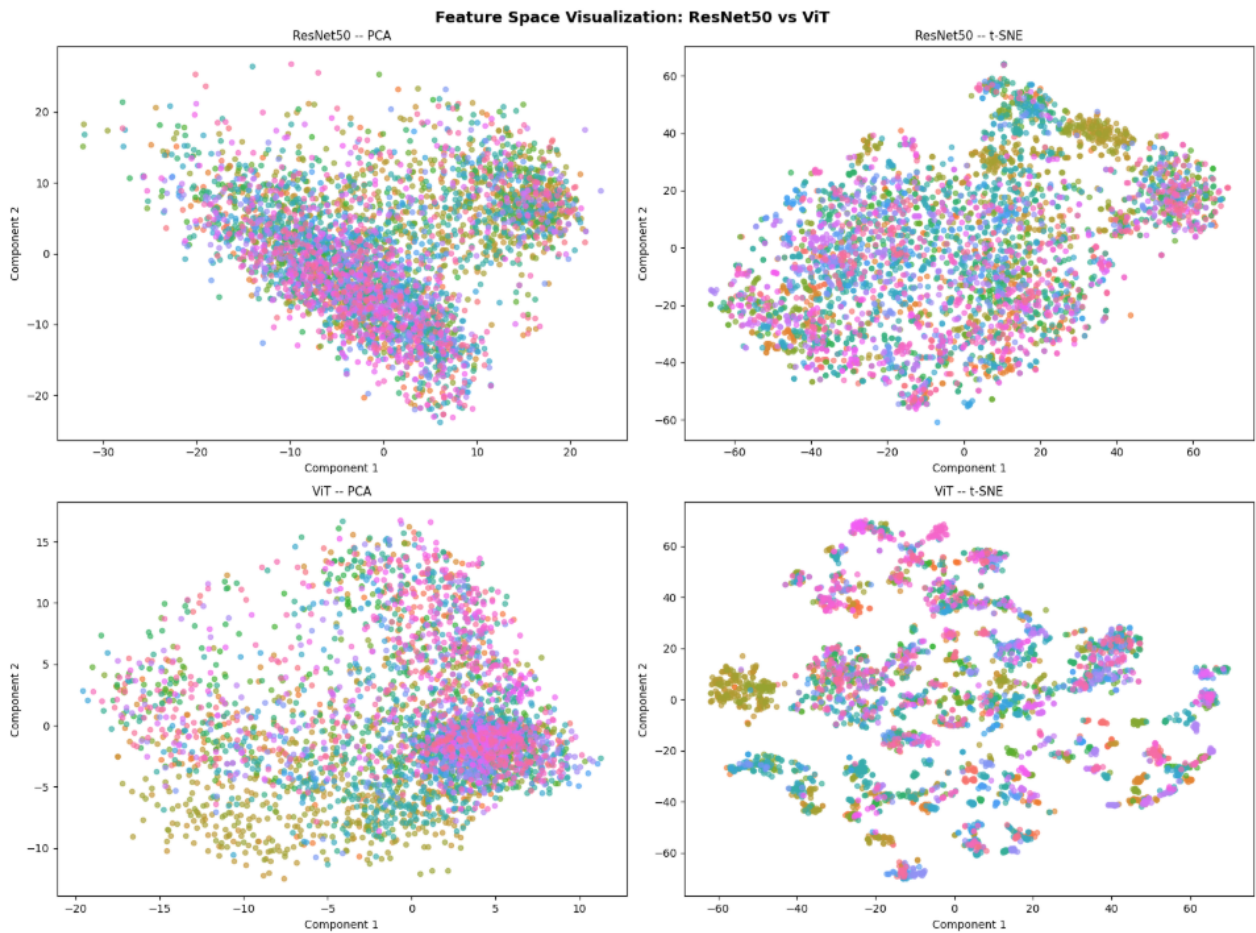
Two factors explain this ranking. First, the ViT [CLS] token captures global image context through self-attention and produces 768-dimensional embeddings that are more linearly separable than the spatially pooled CNN representations. Second, Logistic Regression imposes a simple linear decision boundary that is well suited to a dense pretrained embedding space, while Random Forest's axis-aligned splits fragment the high-dimensional vector and lose information across coordinated dimensions. The same explanation is consistent with the feature-space visualizations and the side-by-side bar chart presented in Section 9.

9 Advanced Analysis and Visualization

This section presents a deeper qualitative and quantitative analysis of the best classifier and its underlying feature space. Four perspectives are considered. Feature space visualization with dimensionality reduction is used to compare the geometry of CNN and Transformer embeddings. The normalized confusion matrix is then computed on the test set to inspect the per-class behavior of the best model. The most-confused class pairs are extracted from the matrix to identify systematic error patterns, and finally a side-by-side bar chart is used to compare all nine model–feature combinations across the four evaluation metrics.

9.1 Feature Space Visualization

To understand why ViT features outperform CNN features, the high-dimensional embedding spaces were projected to two dimensions using two complementary techniques. Principal Component Analysis provides a linear projection that preserves the directions of largest variance, while t-distributed Stochastic Neighbor Embedding provides a non-linear projection that emphasizes local neighborhood structure. For computational efficiency, t-SNE was applied to features that had first been reduced to 50 dimensions with PCA, and a balanced subset of 30 samples per class was sampled before plotting.



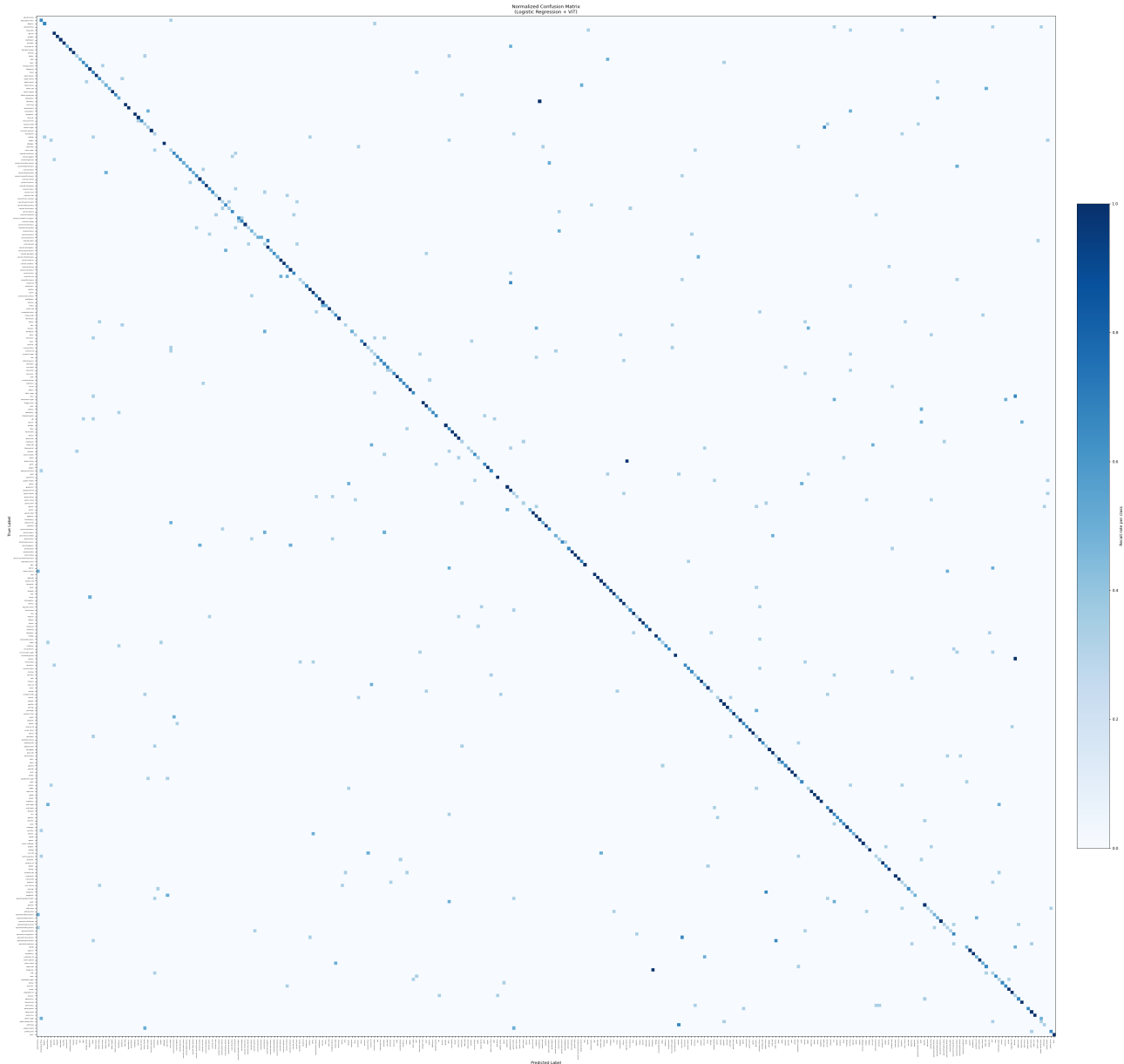
Hình 1: Feature space visualization of ResNet50 and ViT embeddings under PCA (left column) and t-SNE (right column).

Figure 1 reveals a clear difference between the two backbones. The PCA projection of ResNet50 features forms a single elongated blob in which classes overlap heavily. The corresponding ViT projection is more compact and shows weak structure, but the linear projection still cannot separate classes. Under t-SNE, the difference becomes pronounced. ResNet50 produces a dense central cluster with only a few macro-level groupings near the edges, while ViT produces clearly isolated island-like clusters distributed across the projection plane. These t-SNE clusters indicate that ViT places samples of the same ingredient close together in the embedding space and pushes different ingredients apart, which is exactly the structure a downstream classifier needs. This visualization therefore explains why ViT features yield the highest accuracy, precision, recall, and F1 in Table 2.

9.2 Confusion Matrix

The best model identified in Section 8 was used to generate predictions on the 872-sample test set, and a normalized confusion matrix was computed by dividing each row by the number of

true samples in that class. This row normalization makes the diagonal directly interpretable as per-class recall.

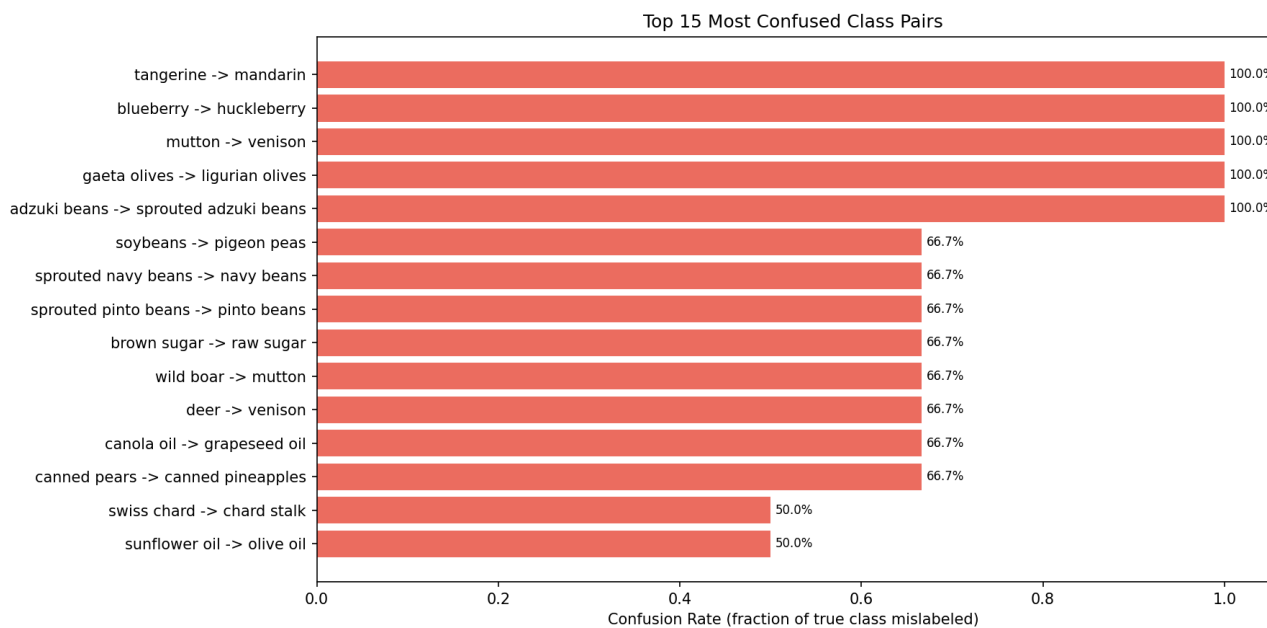


Hình 2: Normalized confusion matrix of Logistic Regression on ViT features. Each row sums to one, and the diagonal represents per-class recall.

The matrix is dominated by a strong blue diagonal, which confirms that the majority of test samples are correctly classified across all 315 classes. Off-diagonal entries are sparse, with most of the matrix appearing white because there are simply no misclassifications between the corresponding pairs. The visible off-diagonal points cluster into a small number of consistent error pairs, which are examined in detail in Section 9.3. Classes with weaker diagonal intensity tend to be those with fewer training samples, reflecting the difficulty of learning a discriminative boundary from a handful of images.

9.3 Error Analysis

To turn the diffuse off-diagonal pattern of the confusion matrix into actionable insight, the per-class confusion rates were ranked and the top fifteen most-confused class pairs were extracted. The ranking is based on the fraction of samples of a true class that were predicted as a specific wrong class.



Hình 3: Top fifteen most-confused class pairs, ranked by confusion rate.

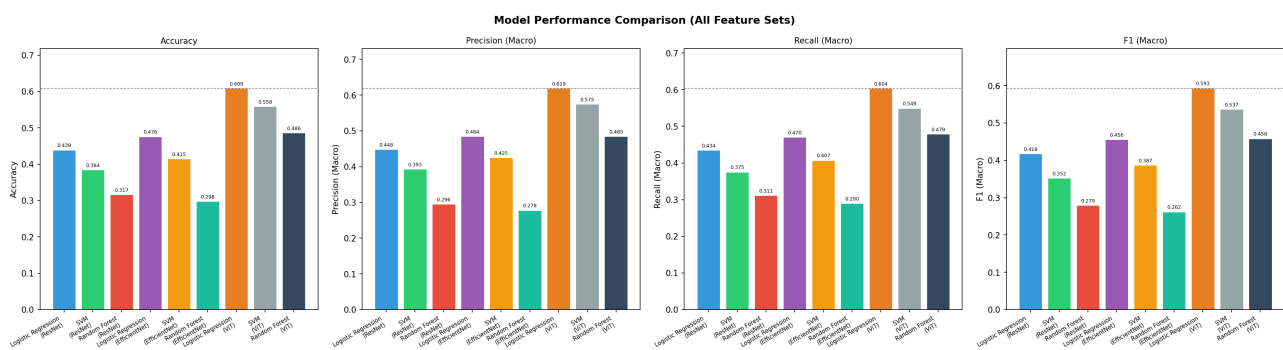
Figure 3 shows several pairs that reach a 100% confusion rate, meaning every test sample of the true class was assigned to the same wrong predicted class. These extreme cases include tangerine misclassified as mandarin, blueberry as huckleberry, mutton as venison, gaeta olives as ligurian olives, adzuki beans as sprouted adzuki beans, and soybeans as pigeon peas. A second tier at 66.7% includes sprouted navy beans as navy beans, brown sugar as raw sugar, deer as venison, and canola oil as grapeseed oil. Two further pairs at 50% are swiss chard with chard stalk and sunflower oil with olive oil.

These errors share two underlying causes. The first is visual similarity. Many of the confused pairs are botanical variants of the same ingredient such as tangerine and mandarin, processing stages of the same product such as raw versus brown sugar or sprouted versus unsprouted beans, or visually near-identical products such as different cooking oils and different game meats. No image-only model can reliably separate these pairs without additional context such as packaging text or scale information. The second cause is class imbalance and low support. The 100% confusion rates appear most often among classes that have very few test samples, where a single misclassified example pushes the rate to its maximum value. The complete ranked

list of confused pairs is stored in `confusion_analysis.csv` for further inspection.

9.4 Model Comparison

To provide a single side-by-side view of all nine combinations across the four evaluation metrics, the values from Table 2 were rendered as grouped bar charts. The dashed horizontal line in each panel marks the highest score, which corresponds to Logistic Regression on ViT features.



Hình 4: Model performance comparison across all nine combinations: accuracy, macro precision, macro recall, and macro F1.

Three patterns are visible across all four panels of Figure 4. The three ViT-based bars consistently occupy the highest positions, with a clear gap of roughly 0.10 to 0.13 between the best ViT result and the best EfficientNet result on each metric. Within each feature group, Logistic Regression scores highest, SVM scores second, and Random Forest scores last, and this ranking is invariant across the four metrics. The gap between ViT and the CNN backbones is largest on precision and F1, which suggests that ViT features not only retrieve the correct class more often but also produce fewer spurious activations on negative classes. The same conclusion held in the feature-space visualization, where tighter clusters under ViT translate directly into stronger discrimination at evaluation time.

10 Discussion

This section interprets the experimental results, compares CNN-based and Transformer-based feature extractors, and discusses the strengths and limitations of the traditional machine learning pipeline established in Sections 5 to 9.

The performance hierarchy observed in Table 2 can be explained primarily by two factors: the quality of the feature representation and the inductive bias of the classifier. Pretrained deep features encode high-level visual information learned from millions of natural images, so the

downstream classifier no longer needs to learn texture or shape primitives from scratch. Once the feature space is rich enough to make classes approximately linearly separable, a simple linear model such as Logistic Regression is sufficient and even preferred. This is exactly what the experiments show. Logistic Regression and SVM, which both rely on margin-based decision boundaries in the embedding space, consistently outperform Random Forest, whose axis-aligned splits do not exploit the joint structure of the high-dimensional vector.

The comparison between CNN and Transformer features is the most informative result of this study. ResNet50 and EfficientNetB0 are convolutional networks that aggregate local receptive fields hierarchically, and their global average pooling layer collapses spatial information into a single descriptor. ViT, in contrast, processes the image as a sequence of patches and uses self-attention to model interactions between any two patches at any distance. The downstream consequence is visible in two places. In the feature-space plots of Section 9.1, ViT under t-SNE produces well-separated clusters, while ResNet50 produces a single dense blob. In the metric table of Section 8.1, ViT features yield approximately 13 to 14 percentage points of additional accuracy over their CNN counterparts when paired with the same classifier. EfficientNetB0 outperforms ResNet50 by a small but consistent margin under linear and kernel classifiers, which is in line with its more recent and parameter-efficient design, but neither CNN closes the gap to ViT.

Several limitations of this study should be acknowledged. The dataset contains 5811 cleaned samples spread across 315 ingredient classes, which gives roughly 19 to 21 samples per class. This low support amplifies class imbalance, because rare ingredients receive only a handful of training images, and the macro-averaged metrics in Section 8 are particularly sensitive to such cases. The error analysis in Section 9.3 also revealed structural difficulty that no amount of additional images would fully resolve. A substantial fraction of the most-confused pairs are botanical variants, processing stages, or near-identical products that cannot be distinguished from a single resized image, for example tangerine and mandarin, or sunflower and olive oil. Resolving these would require either additional input modalities such as text or scale, or a much larger and more diverse image collection.

Despite these limitations, the pipeline has clear strengths. The use of frozen pretrained extractors makes feature extraction a one-time cost. Once `resnet_X_train.npy`, `effnet_X_train.npy`, and `vit_X_train.npy` are written to disk, all downstream experiments operate on compact numerical arrays and run in seconds rather than hours. The shared label files `y_train.npy`, `y_test.npy`, and `label_map.json` enforce alignment across team members and across feature extractors, so the model-feature comparison in Section 8 is a fair comparison under identical labels and splits. The pipeline is also modular, since any new feature extractor can be added

simply by saving its features to a new `.npy` file without modifying the classifier code or the evaluation script. These properties make the pipeline well suited as a baseline against which the end-to-end deep learning approach in Section 11 is compared.

11 Deep Learning Pipeline

This section presents the end-to-end deep learning pipeline developed for image classification after the preprocessing stage. The implementation uses PyTorch together with the Hugging Face Transformers library to fine-tune a Vision Transformer (ViT) model on the prepared dataset. The full workflow includes dataset loading, augmentation, model initialization, fine-tuning, validation-based early stopping, and final testing.

11.1 End-to-End Deep Learning Approach

An end-to-end deep learning approach was implemented to learn discriminative visual features directly from the processed images. Instead of manually designing features, the model receives resized RGB images from the structured training, validation, and test folders and automatically learns the representations required for classification.

The data pipeline uses `ImageFolder` together with `DataLoader` for efficient batch-based loading. The training split is augmented to improve generalization, while the validation and test splits are only normalized to ensure a fair evaluation. This design creates a consistent pipeline from raw processed images to final predictions.

11.2 Architecture Used

The selected architecture is `google/vit-base-patch16-224-in21k`, accessed through `ViTForImageClassification`. Vision Transformer models divide an image into fixed-size patches, embed these patches as tokens, and process them using transformer encoder blocks. This allows the model to capture global relationships across the image more effectively than many conventional convolutional approaches.

In the implemented fine-tuning strategy, most backbone parameters were frozen to preserve the pretrained representation learned from large-scale data. Only the final transformer block, the final layer normalization module, and the classifier head were unfrozen. This partial fine-tuning strategy reduces computational cost while still allowing the model to adapt to the target dataset.

11.3 Training Process

The training pipeline was implemented with the following configuration:

- input image size: 224×224 ;
- training augmentations: random horizontal flip and random rotation of up to 10° ;
- normalization: mean $[0.5, 0.5, 0.5]$ and standard deviation $[0.5, 0.5, 0.5]$;
- batch size: 16 for training, validation, and testing;
- optimizer: Adam with learning rate 10^{-4} and weight decay 10^{-4} ;
- loss function: cross-entropy loss;
- scheduler: `ReduceLROnPlateau`;
- maximum number of epochs: 50;
- early stopping patience: 10 epochs.

During training, the model parameters were updated on the training set and then evaluated on the validation set after each epoch. Four metrics were stored across epochs: training accuracy, validation accuracy, training loss, and validation loss. The best model weights were selected according to the lowest validation loss, which helps reduce overfitting and preserves the most generalizable checkpoint.

The use of early stopping is particularly important in this task. Once the validation accuracy stabilizes and the validation loss no longer improves for several epochs, continued training is unlikely to produce meaningful gains. Therefore, stopping early makes the training process more efficient and reduces the risk of overfitting.

11.4 Results

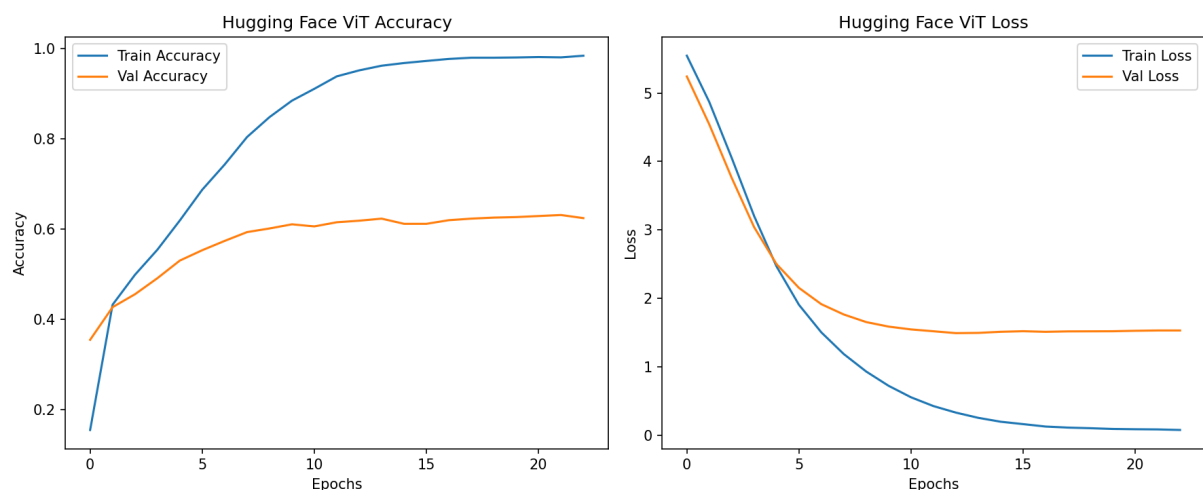
The training behavior of the ViT model is summarized in Figure 5. The recorded curves show that the model learned the training set very effectively, reaching a training loss of 0.0750 and a training accuracy of 0.9835 by epoch 23. However, the validation metrics improved much more slowly and then stabilized after more than 20 epochs.

Near the end of training, the validation loss remained high, with values such as 1.5246, 1.5292, and 1.5290 across the final epochs, while the validation accuracy fluctuated only slightly between 0.6239 and 0.6307. This behavior indicates that the model had already extracted most

of the generalizable patterns available from the training process, and additional epochs were not producing meaningful gains on unseen validation data.

Because the validation performance plateaued, early stopping was triggered before the maximum limit of 50 epochs. This confirms that the patience-based stopping criterion worked as intended by preventing unnecessary training once convergence had effectively been reached.

For the final evaluation, the best saved checkpoint was tested on the test set and achieved a final accuracy of 0.6216. Overall, these results show that the Hugging Face ViT pipeline is capable of learning strong training representations, but the gap between training and validation performance suggests that generalization remains a challenge and may be improved further through additional regularization, data balancing, or hyperparameter tuning.



Hình 5: Training history for the Hugging Face ViT pipeline. The curves show training and validation accuracy together with training and validation loss. The validation accuracy plateaus after more than 20 epochs, so early stopping terminates training before the 50-epoch limit.

12 Comparison: Traditional ML vs Deep Learning

This section compares the traditional machine learning pipeline with the end-to-end deep learning pipeline. The comparison focuses on classification accuracy, computational complexity, processing speed, and the practical strengths and weaknesses of each approach. A key point in interpreting the results is that the dataset is relatively small, with only about 20 samples per class. Under this condition, a deep learning model is less likely to fully demonstrate its true advantage, since transformer-based architectures usually benefit much more from larger and more diverse datasets.

12.1 Accuracy Comparison

The traditional machine learning pipeline was built by extracting pretrained visual features and then training classical classifiers such as Logistic Regression, Support Vector Machine, and Random Forest. Among all traditional ML settings, the best result was obtained by Logistic Regression on ViT features, which achieved an accuracy of 0.6089, a precision of 0.6192, a recall of 0.6037, and an F1-score of 0.5929.

Other traditional ML configurations produced lower accuracies. Logistic Regression with EfficientNet features reached 0.4759, while the corresponding ResNet-based version achieved 0.4392. The SVM models obtained moderate performance, with the best SVM result being 0.5585 on ViT features. Random Forest produced the weakest results overall, with accuracies ranging from 0.3131 to 0.4989.

By comparison, the end-to-end Hugging Face ViT fine-tuning pipeline achieved a final test accuracy of 0.6216. Therefore, the deep learning approach outperformed the best traditional ML baseline by a small margin of approximately 0.0127. Although this improvement is not large, it still indicates that end-to-end fine-tuning can extract slightly more task-specific information than a fixed-feature pipeline.

At the same time, the difference between 0.6216 and 0.6089 is not dramatic. This is consistent with the dataset size: when there are only about 20 samples per class, pretrained feature extraction combined with a lightweight classifier can already perform competitively, while a deep learning model does not yet have enough data to fully exploit its representational capacity.

12.2 Speed and Complexity

From the implementation perspective, the traditional ML pipeline is more modular. It first extracts features using pretrained backbones such as ResNet50, EfficientNetB0, or ViT, saves those features to `.npy` files, and then trains classical classifiers on the extracted vectors. This approach is relatively easy to debug, easy to reuse, and computationally efficient during classifier training, since the extracted features are much smaller than the original images.

The deep learning pipeline is more complex because it performs end-to-end optimization. The model must repeatedly load images, apply augmentations, run the transformer forward pass, compute gradients, and update parameters across many epochs. In this project, the model was trained for more than 20 epochs before early stopping was triggered, which makes the pipeline more computationally expensive than training Logistic Regression or SVM on saved

features.

In terms of speed, traditional ML is usually faster after feature extraction has been completed, especially for small datasets. In contrast, end-to-end deep learning has a higher training cost and requires more careful tuning of learning rate, regularization, augmentation, and stopping criteria. Therefore, the deep learning pipeline has higher implementation and computational complexity, even though it provides slightly better predictive performance.

12.3 Pros and Cons

The traditional ML pipeline offers several advantages. It is easier to interpret, easier to train on limited hardware, and often more stable on small datasets. Since the feature extractor is pretrained and fixed, the risk of overfitting is reduced compared with full or partial fine-tuning. In addition, experimenting with multiple classifiers is straightforward once the features have been saved.

However, traditional ML also has limitations. Because the backbone is used mainly as a frozen feature extractor, the representation is not directly adapted to the target task during classifier training. As a result, some class specific information may be lost, and the final performance may be constrained by the quality of the extracted features.

The deep learning pipeline has the advantage of learning task-specific representations through fine-tuning. This enables the model to adapt pretrained knowledge to the characteristics of the dataset and can improve final accuracy. End-to-end training also provides a more unified framework, since data preprocessing, feature learning, and classification are optimized together.

On the other hand, deep learning is more sensitive to dataset size, hyperparameter selection, and overfitting. In this project, the very high training accuracy together with the lower validation and test accuracy suggests that the model learned the training set strongly but still faced generalization difficulties. This is an important indication that the current dataset is too small for the deep learning approach to show its full power.

Overall, traditional ML is a strong and efficient choice for small datasets, while deep learning becomes more advantageous when more labeled data are available. For the present problem, where each class contains only around 20 samples, the performance gap between the two approaches remains limited. Consequently, the experimental results suggest that dataset scale is a major factor in determining whether end-to-end deep learning can significantly outperform traditional machine learning.

13 Conclusion

This project successfully implemented and evaluated both a traditional machine learning pipeline and an end-to-end deep learning approach for fine-grained food ingredient classification.

Summary of Findings: The experimental results demonstrate that pretrained deep visual features combined with classical classifiers can achieve strong performance, but the choice of feature extractor is critical. Transformer-based features (ViT) significantly outperformed CNN-based features (ResNet50 and EfficientNetB0) by approximately 13 to 14 percentage points in accuracy. This is because the self-attention mechanism in ViT captures global context and creates highly separable clusters in the high-dimensional space, unlike the local receptive fields of CNNs. Furthermore, linear and margin-based classifiers (Logistic Regression, SVM) proved to be much more effective on these dense embeddings compared to tree-based ensembles like Random Forest.

Best Model: In the traditional ML pipeline, the combination of ViT features with Logistic Regression emerged as the best model, achieving an accuracy of 0.6089 and an F1-score of 0.5929. When pushing the limits with an end-to-end deep learning approach, fine-tuning the ViT model yielded the highest overall test accuracy of 0.6216, slightly outperforming the traditional pipeline by a margin of 0.0127.

Key Insights: The relatively small performance gap between the traditional and deep learning pipelines highlights the constraints of the dataset. With only about 20 samples per class, the end-to-end deep learning model showed signs of overfitting (reaching 0.9835 training accuracy while validation plateaued around 0.63). Ultimately, traditional ML remains a highly efficient and robust choice for small datasets, while the full potential of end-to-end deep learning requires a significantly larger scale of labeled data.

14 Future Work

While the current pipelines established a solid foundation for food ingredient classification, several avenues remain for future improvement to address the limitations identified during the error analysis.

- **Dataset Expansion and Balancing:** The most significant bottleneck in the current study is the small dataset size (approximately 20 samples per class). Collecting a larger and more diverse dataset would alleviate class imbalance and provide the end-to-end deep

learning models with sufficient data to generalize better, fully unlocking the representational power of architectures like ViT.

- **Targeted Data Augmentation:** To handle the severe visual similarity in the most-confused class pairs (e.g., botanical variants or processing stages of the same ingredient), targeted data augmentation such as specific color jittering, random cropping, and affine transformations should be applied. This would artificially increase the intra-class variation for rare ingredients.
- **Multi-modal Classification:** Since many misclassified pairs (e.g., tangerine vs. mandarin, or sunflower oil vs. olive oil) are nearly identical in raw image form, future systems should incorporate additional modalities. Integrating text data from packaging, scale information, or contextual metadata would help disambiguate ingredients that visual features alone cannot separate.
- **Advanced Regularization and Feature Ensembling:** For the deep learning pipeline, applying stronger regularization techniques (such as higher dropout rates or label smoothing) could help mitigate the observed overfitting. Additionally, concatenating CNN features (which capture local texture) with Transformer features (which capture global context) could form a robust feature ensemble that leverages the strengths of both architectures.